



태양광 발전량 예측 API 연동

✓ 1. 태양광 발전량 예측 API의 중요성

- 에너지 관리 효율화
 - 발전량 예측을 통해 수요-공급 계획 수립 가능
 - 발전 설비 운영 최적화 가능
- 정확한 기상 데이터 활용
 - 기상청 일사량 데이터를 기반으로 발전량 예측
 - 일사량은 태양광 발전과 밀접한 상관관계
- 데이터 기반 의사결정 지원
 - 예측 데이터는 발전소 운영, 에너지 수급계획 수립에 활용
- 실시간 업데이트
 - 48시간 단위 예측 제공 → 기상 변화에 유연하게 대응 가능
- AI 기술 접목
 - 과거 발전량 + 기상 데이터 기반 AI 모델 활용 → 정확도 향상

✓ 2. 태양광 발전량 예측 API 소개

- 기본 구조: REST API 방식, JSON 반환
- 요소:
 - **Endpoint:** API URL
 - **Method:** GET/POST
 - **Headers:** JSON 등 형식 지정
 - **Parameters:** 지역코드, 기준일 등
- 응답 주요 변수:

변수	의미
fcstDate	예측 날짜 (YYYYMMDD)
fcstTime	예측 시간 (HHMM)
regCd	지역 코드
qgen	예측 발전량 (kWh)

pcap	설비 용량 (kW)
srad	일사량 (W/m^2)
temp	기온 (°C)
wspd	풍속 (m/s)

✓ 3. 웹 훅(Webhook)과 Teams 연동

- **웹 훅이란?**: 이벤트 발생 시 자동으로 외부 URL에 데이터를 전송하는 HTTP 요청
- **Teams 웹 훅 사용 목적:**
 - 실시간 알림: API 실행 결과 즉시 전달
 - 자동화: 반복 작업 생략 가능
 - 유연한 통합: 다양한 시스템과 연결 용이

✓ 4. API 연동 아키텍처

1. **API 서버**: REST API 호출로 예측 데이터 요청
2. **Azure Functions**:
 - API 호출 로직 처리
 - Event Hubs 및 Teams 웹 훅과 연동
3. **Azure Event Hubs**:
 - 대규모 이벤트 스트림 수집
 - 예측 데이터 저장 및 분산 처리
4. **모니터링**:
 - Application Insights를 통한 로그/성능 모니터링
5. **결과 제공**:
 - 전국 단위 예측 정보 제공, 수급계획 등에 활용

✓ 5. 실습 개요: 태양광 발전량 예측 API 연동 실습

| 목표: API + Azure Functions + Event Hubs + Teams 웹훅 연동

✓ 6. Teams 설정

1. 팀 생성 → 채널 생성 → 워크플로 추가
2. 'webhook 수신 시 메시지 게시' 템플릿 선택 → 'Teams 채널 메시지 게시' 액션으로 수정
3. 메시지 내용 설정: 태양광 발전량 예측 수집 스케줄러가 실행되었습니다.

4. `HTTP POST URL` 복사 (Azure Function에서 사용)

✓ 7. Azure Functions 설정

```
// local.settings.json
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "python",
    "SolarEventHubName": "<EventHub 이름>",
    "SolarEventHubConnectionString": "<EventHub 연결문자열>",
    "AzureWebHookUrl": "<복사한 웹훅 URL>"
  }
}
```

```
# 필수 명령어
pip install -r requirements.txt
azurite start
```

✓ 8. 함수 실행 (로컬)

1. `solar_predict_event_scheduler` 함수 실행
2. 터미널 로그 확인
3. Teams 웹훅 메시지 확인

✓ 9. Event Hubs 데이터 확인

1. Azure Portal 접속 → Event Hub 네임스페이스 접속
2. `Data Explorer` → `이벤트 보기` 선택
3. 예측 데이터 본문 확인: `{ "city": "경기도", "qgen": ..., ... }`

✓ 10. Azure에 Function App 생성 및 배포

- 생성 옵션:
 - Hosting Plan: Consumption
 - Runtime: Python 3.11
 - Resource Group, Storage Account, App Insight 새로 생성
- 배포:

- VS Code에서 `Deploy to Function App...`
- 설정 업로드 포함

✓ 11. 클라우드에서 실행 및 확인

1. Azure Portal에서 Function App 선택
2. `solar_predict_eventhub_scheduler` → `Execute Function Now...`
3. 로그 확인 + Teams 웹훅 메시지 확인
4. Event Hub에서 데이터 수집 여부 재확인

✓ 12. 흐름 요약

1. Azure Functions를 이용한 태양광 발전량 API, Teams Webhook 연동
2. Azure Portal을 이용한 Azure Functions 생성
 - a. 리소스 그룹 생성
 - b. 마켓 플레이스에서 Function App 검색 후 리소스 생성
 - c. 호스팅 플랜 선택: 사용량 혹은 유연한 사용량
 - d. 함수 앱 이름, 운영체제(Linux), 런타임 스택: Python, 버전: 3.11, 지역: East US(quota 에러 발생시 East US2)
3. Azure Function App 배포
 - a. VS Code에서 명령 팔레트 실행
 - b. Azure Functions: Deploy to Function App... 명령 실행
 - c. 배포 후 환경 변수 업로드
4. Azure Function App 실행
 - a. 웹 브라우저에서 배포한 Azure Functions 화면에서 `solar_predict_eventhubs_scheduler` 함수를 클릭한 뒤 로그 탭을 클릭(함수 실행 모니터링)
 - b. VS Code로 돌아간 뒤 좌측 패널에서 Azure 아이콘 클릭
 - c. Resources 탭에서 자기가 만든 Azure Function App 클릭
 - d. Functions 클릭 후 `solar_predict_eventhubs_scheduler` 함수 실행
 - e. 주의 사항: 원격에 배포 된 Azure Function App을 실행하는 것이기 때문에 자기 VS Code 터미널에 함수 실행 로그가 나타나면 안됨(로그 예: 태양광 예측 데이터 수집 스케줄러가 시작되었습니다.)
5. 함수 실행 결과 확인
 - a. 4-1에서 발생한 로그를 확인

- b. 함수 실행이 잘 되었다면 eventhub를 실행 (웹 브라우저에서 탭을 2개(Azure Function App, Azure EventHub) 켜놓고 있기를 권장)
- c. eventhub 페이지에서 Data Explorer 메뉴 클릭
- d. 이벤트 보기 버튼을 클릭하여 태양광 발전량 예측 데이터가 들어온 것을 확인

태양광 function_app.py 전체코드

```
import logging
import azure.functions as func
import pandas as pd
import datetime
from datetime import timezone
import pytz
import requests
import json
import os

app = func.FunctionApp()

@app.timer_trigger(schedule="0 0 0 * * *", arg_name="timer", run_on_startup=False, u
@app.event_hub_output(arg_name="event", event_hub_name=os.environ["SolarEventH
def solar_predict_eventhubs_scheduler(timer: func.TimerRequest, event: func.Out[str])
    logging.info('태양광 예측 데이터 수집 스케줄러가 시작되었습니다.')

    df = pd.read_csv("./data/solar_city.csv", encoding="euc-kr")

    utc_timestamp = datetime.datetime.now(tz=timezone.utc)
    kst = pytz.timezone('Asia/Seoul')
    kst_timestamp = utc_timestamp.astimezone(kst)
    base_date = kst_timestamp.strftime("%Y%m%d")

    url = "https://bd.kma.go.kr/kma2020/energy/energyGeneration.do"
    headers = {"Content-Type": "application/x-www-form-urlencoded"}
    solar_data_list = []
    skipped_index = []

    for index, row in df.iterrows():
        if pd.isna(row["격자X"]) or pd.isna(row["격자Y"]) or pd.isna(row["행정구역코드"]) or
            skipped_index.append(index)
            continue
```

```

try:
    reg_cd = row["행정구역코드"]
    city = row["도"]
    county = row["시"]
    lat = row["위도(초/100)"]
    lon = row["경도(초/100)"]

    payload = f"baseDate={base_date}&fcstDate={base_date}&fcstTime=0000&reg
    response = requests.post(url, headers=headers, data=payload)
    response.raise_for_status()

    data = response.json()
    items = data['result']

    for item in items:
        solar_data = {
            "city": city,
            "county": county,
            "fcstDate": item["fcstDate"],
            "fcstTime": item["fcstTime"],
            "pcap": item["pcap"],
            "qgen": item["qgen"],
            "regCd": item["regCd"],
            "srad": item["srad"],
            "temp": item["temp"],
            "wspd": item["wspd"],
            "lat": lat,
            "lon": lon
        }
        solar_data_list.append(solar_data)

except (ValueError, TypeError) as e:
    logging.error(f"Failed to convert 격자X or 격자Y to integer in row {index}: {e}")

logging.info(f"태양광 예측 데이터 수집이 완료되었습니다. 수집된 데이터: {len(solar_data_list)}")
event.set(solar_data_list)

# ✅ Teams Webhook 메시지를 변수 기반 단순 텍스트로 전송
if os.environ.get("AzureWebHookUrl") is not None:
    logging.info("환경 변수에 웹훅 URL이 설정되어있습니다. 웹훅을 요청합니다.")
    webhook_url = os.environ["AzureWebHookUrl"]

```

```

if len(solar_data_list) > 0:
    df_solar = pd.DataFrame(solar_data_list)
    total_count = len(solar_data_list)
    df_solar["temp"] = df_solar["temp"].astype(float)
    city_temp = df_solar.groupby("city")["temp"].mean().round(1)
    city_temp_lines = "<br>".join([f"- {city}: {temp}°C" for city, temp in city_temp.items()])
    max_srad_row = df_solar.loc[df_solar["srad"].astype(float).idxmax()]
    top_location = f'{max_srad_row["city"]} {max_srad_row["county"]}'
else:
    total_count = 0
    city_temp_lines = "데이터 없음"
    top_location = "없음"

# Teams로 보낼 메시지 텍스트
message_text = (
    f"☀️ 태양광 예측 데이터 수집이 완료되었습니다.<br>"
    f"- 수집 건수: {total_count}건<br>"
    f"- 최고 일사량 지역: {top_location}<br>"
    f"- 도별 평균 기온:<br>{city_temp_lines}"
)

# ✅ 단순 text 메시지로 전송
response = requests.post(
    webhook_url,
    headers={"Content-Type": "application/json"},
    json={"text": message_text}
)
logging.info(f"웹훅 실행 결과: {response.status_code}")
response.raise_for_status()
else:
    logging.info("웹훅 URL이 설정되어 있지 않습니다. 웹 훅 요청을 생략합니다.")

logging.info(f"{skipped_index}번 행은 데이터 수집에서 제외되었습니다.")
logging.info('태양광 예측 데이터 수집 스케줄러가 종료되었습니다.')

```

팀즈 workflow 세팅

```

<p>@{triggerBody()?['text1']}<br>
@{triggerBody()?['text2']}<br>
@{triggerBody()?['text3']}</p>

```